

数据湖集成 — Kafka 数据格式说明

本文档供数据湖厂家集成对接，说明推送至 Kafka 的消息格式、字段含义及消费方式。

1. Kafka 连接信息

项目	说明
Brokers	10.81.151.147:21007, 10.81.151.148:21007, 10.81.151.149:21007
认证方式	Kerberos / GSSAPI (SASL)
序列化	Key 和 Value 均为纯文本 JSON (UTF-8)

1.1 Kafka Topic 命名

每个电厂使用独立 Topic，命名格式 `gyhlw_{plant_code}`，全小写下划线连接：

电厂	编码	Topic 名称
黄岛电厂	sd02001	gyhlw_sd02001
东营电厂	sd03001	gyhlw_sd03001
郓城电厂	sd04001	gyhlw_sd04001
鲁北电厂	sd05001	gyhlw_sd05001
滨州电厂	sd06001	gyhlw_sd06001
临清电厂	sd07001	gyhlw_sd07001

2. 消息语义

MQTT 侧接收到的是一条包含多个测点的批量消息，服务端将其拆分为逐测点单条写入 Kafka：

- 每 1 条 Kafka 消息 = 1 个测点的 1 次采样
- 消息 Key = 测点名 (`tag_name`)，确保同一测点写入同一分区，保证该测点数据的时序有序
- 消息 Value = 下方 JSON

3. 设计说明：为什么逐测点单条而非合并为数组

常见疑问：MQTT 侧一次收到的是包含多个测点的批量消息，为什么不直接整包写入 Kafka，而要拆成逐条？

考量	逐测点单条 Record（当前方案）	合并为数组写入一条 Record
分区有序	同一 <code>tag_name</code> 天然按 Key Hash 进入同一分区，严格有序	无法按测点分区，同一测点数据可能散落不同分区，顺序无保证
消费并行度	多个分区可并发消费，吞吐高	单分区内消费，并行度受限于分区数
消费端处理	每条直接 <code>JSON.parse()</code> 即可，无需拆包	需解析外层数组、遍历内层元素，增加逻辑复杂度
失败重试	单条失败不影响其他测点	需自行实现部分成功/失败的回退逻辑
消息大小	每条几百字节，可预测，网络/存储友好	数百测点合并后可能达数十 KB，增加 IO 开销
过滤灵活性	消费端可按需订阅特定测点（配合点表白名单）	无法细粒度过滤，必须全量拉取

结论：逐测点单条 Record 是 Kafka 场景的标准实践——传输层通过 `SendMessage` 批量提交（一次网络往返），消息层保持单测点粒度，兼顾吞吐、有序性和消费灵活性。

4. Kafka Value 格式

3.1 JSON 示例

```
{
  "tag_name": "DDM.SIS.1DCS_10DCS_TDM_01",
  "device_id": "sis-collect-dev-dy",
  "value": 26.5,
  "timestamp": 1600397414,
  "state": 1
}
```

3.2 字段说明

字段	类型	必填	说明
<code>tag_name</code>	string	是	测点名称，全局唯一。也是这条消息的 Kafka Key
<code>device_id</code>	string	是	设备标识，标识数据来源设备
<code>value</code>	double	是	测点数值
<code>timestamp</code>	int64	是	采集时间戳， Unix 秒级 （如 <code>1600397414</code> = 2020-09-18 02:50:14 UTC）
<code>state</code>	int	是	质量码： <code>1</code> = 良好 (Good)， <code>0</code> = 异常 (Bad)

4. Kafka Message Key

- 格式：测点名 (`tag_name`) 的 UTF-8 字节
- 作用：利用 Kafka 的 Hash 分区策略，同一测点始终路由到同一分区，保证消费时该测点数据严格有序
- 消费端无需特殊处理，直接读取即可；如业务上需要按设备聚合，可按 `device_id` 二次分组

5. 消息示例（多条连续记录）

以 Topic `gyhlw_sd03001`（东营电厂）为例，可能连续出现的 3 条消息：

```
{ "tag_name": "DDM.SIS.1DCS_T0101", "device_id": "sis-collect-dev-dy", "value": 23.1, "timestamp": 1600397002, "state": 1 }
{ "tag_name": "DDM.SIS.1DCS_T0102", "device_id": "sis-collect-dev-dy", "value": 45.7, "timestamp": 1600397002, "state": 1 }
{ "tag_name": "DDM.SIS.1DCS_T0103", "device_id": "sis-collect-dev-dy", "value": 12.0, "timestamp": 1600397003, "state": 0 }
```

6. 消费注意事项

1. 编码：Key 和 Value 均为 UTF-8 字符串，Value 为合法 JSON，可直接 `JSON.parse()` 解析
2. 时序：同一 `tag_name` 的消息保证分区内有序；不同测点之间不保证全局时序
3. 去重：本服务使用 `At-Least-Once` 语义（QoS 1），极端情况下可能存在重复消息，建议消费端按 `tag_name + timestamp` 做幂等去重
4. 数据密度：消息频率取决于现场设备采集频率，高峰期每秒可达数百条
5. 质量码：`state=0` 的记录仍会写入 Kafka，消费端可根据业务需求过滤或标记